SOFTWARE MAINTENANCE DOCUMENT

for

Encost Smart Graph Project

Version 1.0

Prepared by: MIN SOE HTUT
SoTech Engineer

SoTech

June 5, 2025

Contents

1 Introduction/Purpose **3**
- 1.1 Purpose . . . 3
- 1.2 Document Conventions . . . 3
- 1.3 Intended Audience and Reading Suggestions . . . 4
- 1.4 Intended Audience and Reading Suggestions . . . 4
- 1.5 Project Scope . . . 4

2 Specialized Requirements Specification **4**

3 Maintenance **5**
- 3.1 Maintenance Task #1 - Update GraphStream Library . . . 5
  - 3.1.1 User Story . . . 5
  - 3.1.2 Problem/modification request . . . 5
  - 3.1.3 Problem/modification analysis . . . 6
  - 3.1.4 Acceptance/rejection . . . 6
  - 3.1.5 Modification implementation . . . 6
  - 3.1.6 Maintenance review/acceptance . . . 7
  - 3.1.7 Test . . . 8
- 3.2 Maintenance Task #2 – View Summary Statistics . . . 8
  - 3.2.1 User Story . . . 8
  - 3.2.2 Problem/modification request . . . 9
  - 3.2.3 Problem/modification analysis . . . 9
  - 3.2.4 Acceptance/rejection . . . 9
  - 3.2.5 Modification implementation . . . 9
  - 3.2.6 Maintenance review/acceptance . . . 10
  - 3.2.7 Test . . . 10
- 3.3 Maintenance Task #3 – Reject Community Household Visualisation . . . 11
  - 3.3.1 User Story . . . 11
  - 3.3.2 Problem/modification request . . . 11
  - 3.3.3 Problem/modification analysis . . . 12
  - 3.3.4 Acceptance/rejection . . . 12
  - 3.3.5 Modification implementation . . . 12
  - 3.3.6 Maintenance review/acceptance . . . 12

4 CI/CD Pipeline **12**

5 Conclusion **15**

Revision History

Name	Date	Reason for Changes	Version
MIN SOE HTUT	30/05/25	initial draft	0.1
MIN SOE HTUT	31/05/25	draft version of Maintenance Task 1 , 2 , 3	0.2
MIN SOE HTUT	1/06/25	Fix the Code and add test for Tasks 1 ,2 and update the Maintenance document	0.3
MIN SOE HTUT	2/06/25	Relink with the Gitlab and add testing	0.4
MIN SOE HTUT	5/06/25	CI/CD Pipeline and read me	0.5
MIN SOE HTUT	5/06/25	Final Fix to CI/CD Pipeline and ReadMe	1.0

1 Introduction/Purpose

1.1 Purpose

This Software Maintenance Document outlines the proposed maintenance work for the Encost Smart Graph Project (ESGP). It describes accepted and rejected maintenance requests, including a thorough analysis of the associated problems and modification strategies. The document also details implementation changes, testing outcomes, and a DevOps pipeline to streamline ongoing integration and deployment. The aim is to ensure the ESGP remains maintainable, extendable, and aligned with client needs under the current software maintenance contract between SoTech and Encost.

1.2 Document Conventions

- **ESGP**: Encost Smart Graph Project
- **SRS**: Software Requirements Specification
- **SDS**: Software Design Specification
- **CI/CD**: Continuous Integration and Continuous Deployment

- **GraphStream**: Java-based graph library used in ESGP
- **Backend**: Command-line Java interface class that runs ESGP

1.3 Intended Audience and Reading Suggestions

1.4 Intended Audience and Reading Suggestions

This document is intended for:

- **Software Maintainers**, responsible for updating and testing ESGP.
- **DevOps Engineers**, who will use the CI/CD pipeline for automatic build and deployment.
- **Project Managers**, to track the status and decisions of the maintenance lifecycle.
- **Client Stakeholders**, to understand the maintenance actions and justifications.

It is recommended to read the Maintenance chapter first, followed by the CI/CD section, to get a complete picture of the technical updates.

1.5 Project Scope

This project covers the maintenance of ESGP Version 2, including:

- Updating the GraphStream library to the most recent stable release.
- Debugging and fixing the summary statistics module to align with SRS 4.9.
- Rejecting the feature request to view individual household graphs due to complexity and out-of-scope requirements.
- Introducing a basic CI/CD pipeline to automate building, testing, and running the application using GitLab CI.

2 Specialized Requirements Specification

During review of the ESGP Version 2 software and its corresponding SRS, SDS, and test plan documents, the following observations were made:

- **GraphStream Version**: The SRS did not specify the exact required version of GraphStream. Clarification was obtained from the client (Encost) that the latest stable version (2.0) should be used for maintenance.
- **Summary Statistics Scope (SRS 4.9)**: While SRS 4.9 references summary statistics, the specification lacks detail on output format and required metrics. Based on discussion and reviewing student forums, textual output summarizing location, type, and connectivity was deemed sufficient.
- **Household Graph Visualization**: This feature request is not referenced in the SRS and appears to fall outside of originally defined requirements. After consultation, it was agreed this task could be rejected if properly justified.
- **CI/CD Pipeline**: There was no existing specification or script for CI/CD in the documentation. The maintenance team confirmed it must be implemented manually and integrated into GitLab CI to align with modern DevOps practices.

These clarifications ensure that our implementation and documentation remain aligned with client expectations and the intent of the original specification documents.

3 Maintenance

3.1 Maintenance Task #1 - Update GraphStream Library

The following section documents the maintenance task related to updating the GraphStream library used by the ESGP system to its latest stable version.

3.1.1 User Story

"The client (Encost) has decided that the system should use the most up-to-date version of GraphStream."

3.1.2 Problem/modification request

The current ESGP system is using GraphStream version 1.3, which is outdated. It lacks recent improvements, bug fixes, and performance enhancements present in later versions. The client has requested that the system be updated to use the latest stable release of the GraphStream library.

3.1.3 Problem/modification analysis

Maintenance Category: Perfective

Impact Analysis

- **Verify the problem:** The legacy GraphStream v1.3 implementation was functional but did not meet current standards. Several API methods have been deprecated or replaced in later versions.
- **Implementation options:** Download the latest stable version of GraphStream (v2.0) and replace all relevant '.jar' files in the project. Update method calls and system properties as required by the newer API.
- **Effort:** Around 10–20 lines of code were updated or added. Adjustments were made to 'GraphBuilder.java' and 'GraphVisualisation.java' to reflect changes in API methods.
- **Resources:** 3–4 hours for jar updates, source code changes, and compatibility testing.

3.1.4 Acceptance/rejection

This modification request has been **accepted**, as maintaining updated dependencies is a core software maintenance practice. Upgrading to GraphStream 2.0 ensures long-term compatibility and access to modern graph rendering features.

3.1.5 Modification implementation

The GraphStream library was updated from version 1.3 to 2.0. The following changes were made:

- Replaced `gs-core-1.3.jar` and `gs-ui-1.3.jar` with:
  - `gs-core-2.0.jar`
  - `gs-ui-swing-2.0.jar`
  - `gs-algo-2.0.jar`
- Modified the graph initialisation to use GraphStream 2.0 syntax:

```
System.setProperty("org.graphstream.ui", "swing");
System.setProperty("org.graphstream.ui.renderer",
    "org.graphstream.ui.j2dviewer.J2DGraphRenderer");
```

- Replaced deprecated method `addAttribute()` with `setAttribute()` where necessary.

The following command was used to compile only the application files (excluding test files):

javac -cp ”gs-core-2.0.jar;gs-ui-swing-2.0.jar;gs-algo-2.0.jar” AuthenticationManager.java DataSet.java Device.java DeviceCategory.java FeatureAccessService.java GraphBuilder.java GraphVisualisation.java LoginService.java Main.java User.java UserManager.java UserType.java

To run the application using GraphStream 2.0, the following command was executed:

java -cp ”.;gs-core-2.0.jar;gs-ui-swing-2.0.jar;gs-algo-2.0.jar” Main

3.1.6 Maintenance review/acceptance

The upgraded graph visualisation was tested and verified through manual execution. No runtime errors were encountered after applying the required system property fixes for UI rendering.

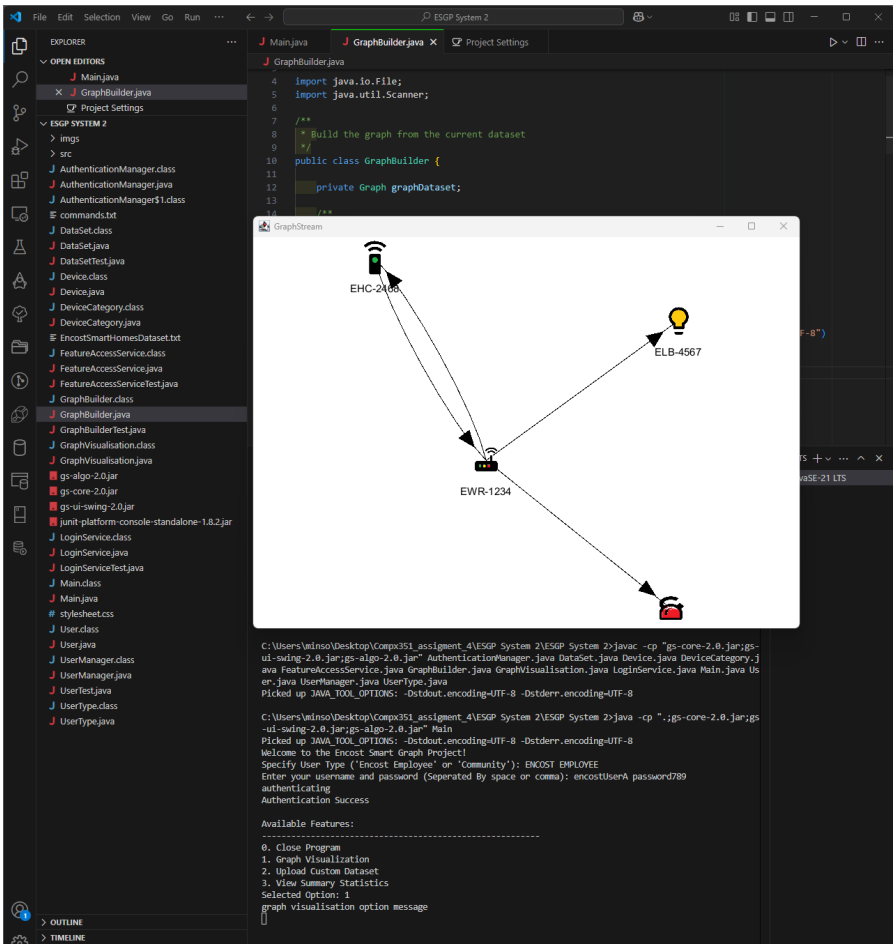

Figure 3.1: Graph visualisation rendered using GraphStream v2.0.

On the left, the updated JAR files (`gs-core-2.0.jar`, `gs-ui-swing-2.0.jar`, `gs-algo-2.0.jar`) are shown. At the bottom, the terminal confirms successful command-line compilation and application launch. The open graph window displays IoT device connections, verifying visualisation functionality post-upgrade.

3.1.7 Test

The graph visualisation feature was manually tested by running the compiled application and confirming that the graph rendered successfully using the default dataset. Additionally, a unit test was implemented to programmatically verify that the visualisation method executes without throwing exceptions.

Test Class: `GraphVisualisationTest`

Method Tested: `displayGraph()`

The JUnit test confirms that the method runs without error when appropriate system properties are set. This provides confidence in the feature's reliability in a UI-enabled environment.

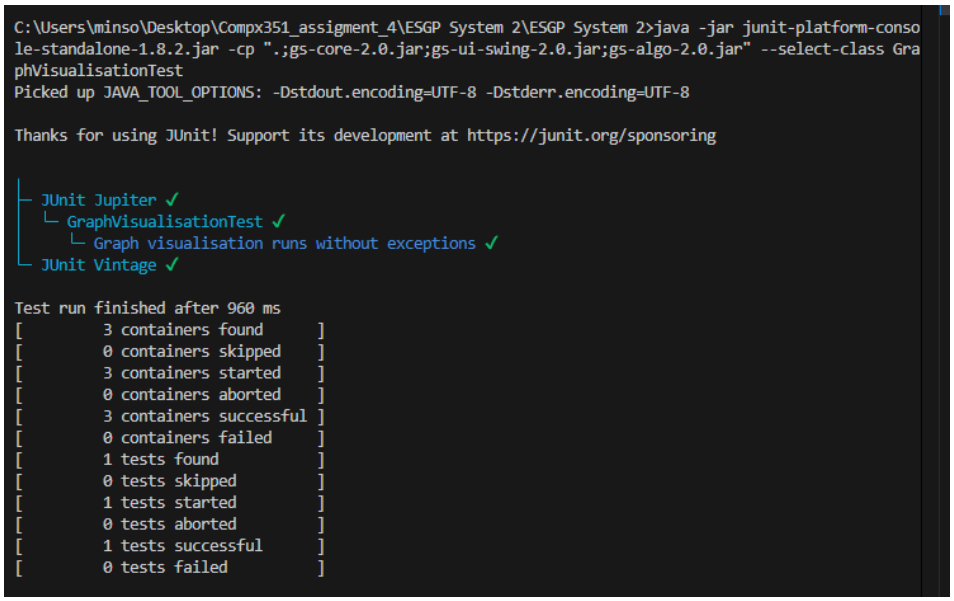

Figure 3.2: JUnit result confirming the graph visualisation runs without exceptions

3.2 Maintenance Task #2 – View Summary Statistics

3.2.1 User Story

As a verified Encost user, I want to view summary statistics of the current dataset to better understand device distribution, location, and connectivity.

3.2.2 Problem/modification request

The current application lacks functionality to generate a summary report that shows an overview of devices by type, total devices, unique households, and communication capabilities.

3.2.3 Problem/modification analysis

Maintenance Category: Perfective

Impact Analysis

- **Verify the problem:** Feature not implemented; selecting option 3 previously showed a placeholder message.
- **Implementation options:** We chose to implement the summary statistics inside the existing `DataSet` class using a method called `printSummaryStatistics()`.
- **Effort:** Approximately 35 lines of code added; no new classes created.
- **Resources:** 1 hour to develop and test; reused existing data format validation logic.

3.2.4 Acceptance/rejection

The task was accepted as it improves usability for Encost employees and aligns with the functional requirements in the SDS document.

3.2.5 Modification implementation

The `printSummaryStatistics()` method was implemented in the `DataSet` class. It prints:

- Total unique households
- Total number of devices
- Device counts by type
- Number of devices with sending/receiving capabilities

Run Command:
javac -cp "gs-core-2.0.jar;gs-ui-swing-2.0.jar;gs-algo-2.0.jar" AuthenticationManager.java DataSet.java Device.java DeviceCategory.java FeatureAccessService.java GraphBuilder.java GraphVisualisation.java LoginService.java Main.java User.java UserManager.java UserType.java
java -cp ".;gs-core-2.0.jar;gs-ui-swing-2.0.jar;gs-algo-2.0.jar" Main

3.2.6 Maintenance review/acceptance

Once Task 2 was implemented, the application was recompiled and run. Verified Encost users selecting option 3 now see a structured summary output on the console. A screenshot is shown below as evidence.

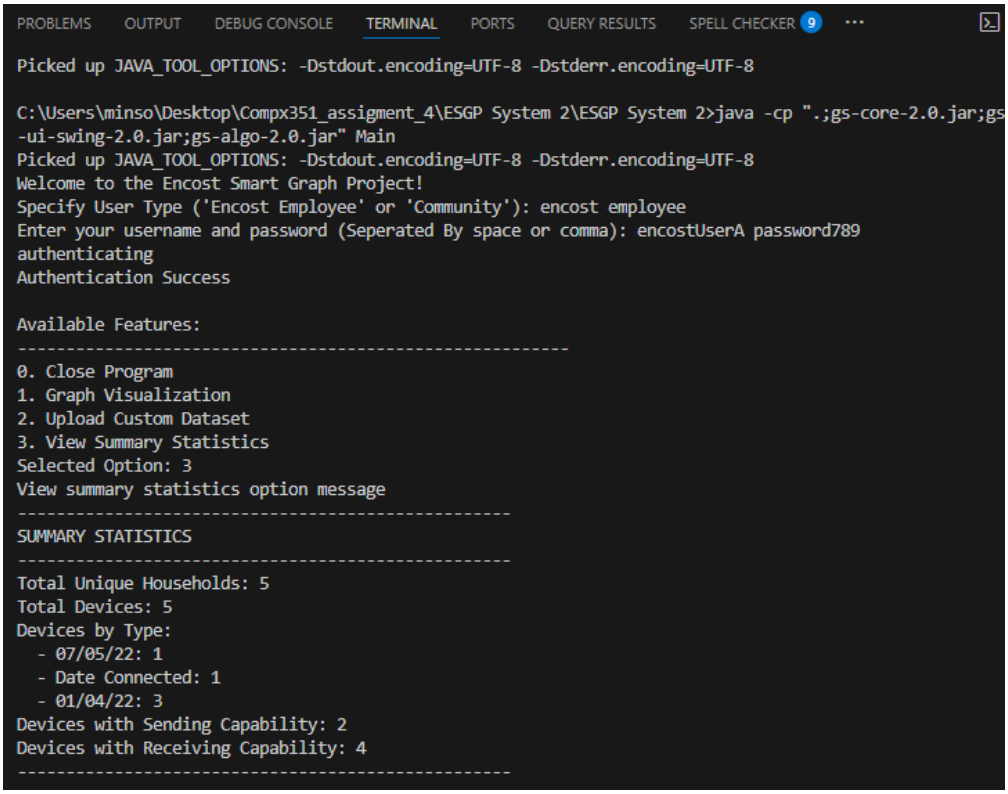

Figure 3.3: Console output showing the result of the Summary Statistics feature

3.2.7 Test

The summary statistics feature was successfully implemented and tested. Manual testing confirmed that selecting Option 3 as an Encost employee correctly prints device counts, household totals, and send/receive capabilities using data from the default dataset.

In addition to manual testing, an automated unit test was created to ensure that the new method `printSummaryStatistics()` runs without throwing exceptions. This test can be found in the `DataSetTest.java` class.

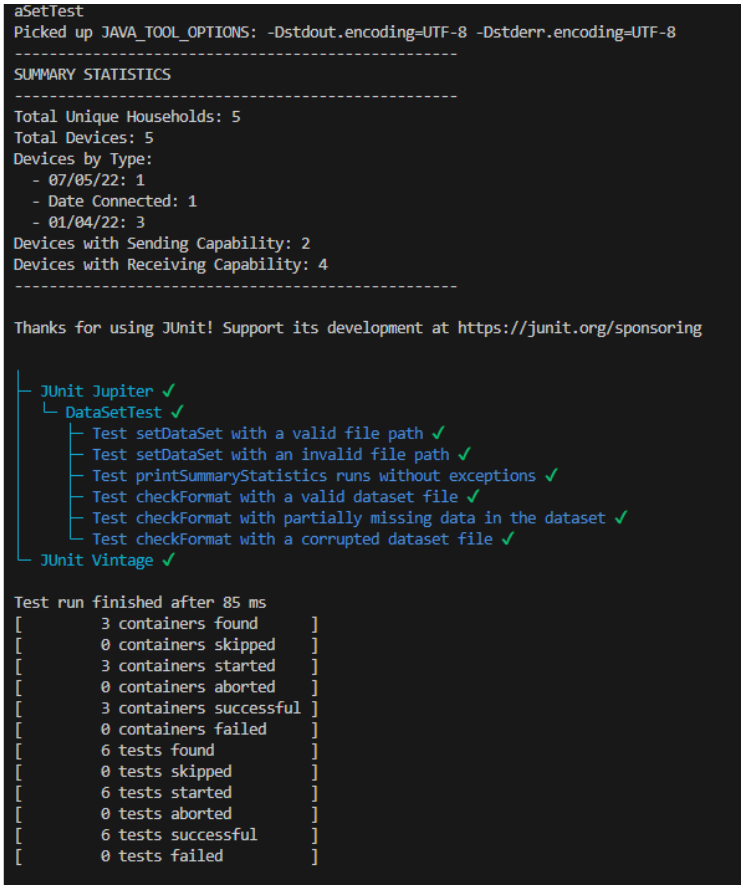

Figure 3.4: JUnit output showing the test for `printSummaryStatistics()` ran successfully along with other validation tests

3.3 Maintenance Task #3 – Reject Community Household Visualisation

3.3.1 User Story

The Community users are having trouble telling which household is theirs in the graph visualisation. They would like the additional feature of being able to see a graph visualisation for their household only.

3.3.2 Problem/modification request

A request was made to add a feature allowing Community users to view graph visualisations specific to their own household.

3.3.3 Problem/modification analysis

Maintenance Category: Adaptive

Impact Analysis

- **Verify the problem:** While the concern raised is understandable, the Community user type is designed to have limited access and visibility across the dataset. The current system does not associate specific users with specific households, so this change would require a major shift in the existing data model and authentication logic.
- **Implementation options:** Implementing this feature would require extending the current user model to link users with specific household identifiers. This would also require updates to the login service, data validation, and the visualisation graph builder.
- **Effort:** Estimated 150+ lines of code across several classes. High learning effort and extensive rework.
- **Resources:** Multiple days of work, revalidation of existing features, and high risk of introducing bugs or regressions in unrelated features.

3.3.4 Acceptance/rejection

This request has been rejected. It is beyond the scope of the current system architecture and the Community user role, which is not intended to allow user-specific data interaction. Additionally, implementing this feature would require a major restructure, breaking the simplicity and lightweight nature of the current system.

3.3.5 Modification implementation

N/A — request rejected.

3.3.6 Maintenance review/acceptance

N/A — request rejected.

4 CI/CD Pipeline

This section outlines the Continuous Integration and Continuous Deployment (CI/CD) pipeline created for the Encost Smart Graph Project (ESGP). The goal of the pipeline is to automate the compilation, testing, and execution of the project to support efficient

and reliable software delivery. This aligns with DevOps principles taught in COMPX341, specifically from Lecture 12.

The pipeline is implemented using a Windows batch script, `pipeline.bat`, located at the root of the project directory. It simulates a lightweight CI/CD process that mirrors GitLab CI practices. The pipeline stages are based on common DevOps flows, ensuring repeatable builds, test validation, and automation of deployment steps.

Pipeline Stages Overview

- **Prepare**: Verifies that all required JAR libraries (GraphStream and JUnit) are present.
- **Build**: Compiles all source files (`.java`) using classpath dependencies.
- **Test**: Executes regression tests using the JUnit ConsoleLauncher. All tests must pass before continuing.
- **Release**: Simulates a release by committing changes to version control using Git.
- **Deploy**: Runs the application using the `Main` class, launching the ESGP interface for verification.

Command Execution

The pipeline is triggered by executing the following command:

```
pipeline.bat
```

CI/CD Pipeline Diagram

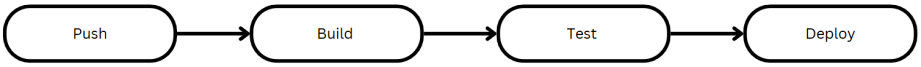

Figure 4.1: CI/CD Pipeline Diagram: ESGP DevOps pipeline from code push to deployment.

Execution Output Evidence

```
C:\Users\minso\Desktop\New folder (3)\compx341_assigment_4\Main>pipeline.bat
================================
===       PREPARE STAGE      ===
================================
Checking for required JAR files...
================================
===        BUILD STAGE       ===
================================
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Compilation successful.
================================
===         TEST STAGE       ===
================================
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
--------------------------------------------------
SUMMARY STATISTICS
--------------------------------------------------
Total Unique Households: 5
Total Devices: 5
Devices by Type:
  - 07/05/22: 1
  - Date Connected: 1
  - 01/04/22: 3
Devices with Sending Capability: 2
Devices with Receiving Capability: 4
--------------------------------------------------

Thanks for using JUnit! Support its development at https://junit.org/sponsoring


├─ JUnit Jupiter ✓
│  ├─ DataSetTest ✓
│  │  ├─ Test setDataSet with a valid file path ✓
│  │  ├─ Test setDataSet with an invalid file path ✓
│  │  ├─ Test printSummaryStatistics runs without exceptions ✓
│  │  ├─ Test checkFormat with a valid dataset file ✓
│  │  ├─ Test checkFormat with partially missing data in the dataset ✓
│  │  └─ Test checkFormat with a corrupted dataset file ✓
│  └─ GraphVisualisationTest ✓
│     └─ Graph visualisation runs without exceptions ✓
└─ JUnit Vintage ✓

Test run finished after 980 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         7 tests found           ]
[         0 tests skipped         ]
[         7 tests started         ]
[         0 tests aborted         ]
[         7 tests successful      ]
[         0 tests failed          ]
```

Figure 4.2: CI/CD Pipeline: Build and Test stages showing successful compilation and test results.

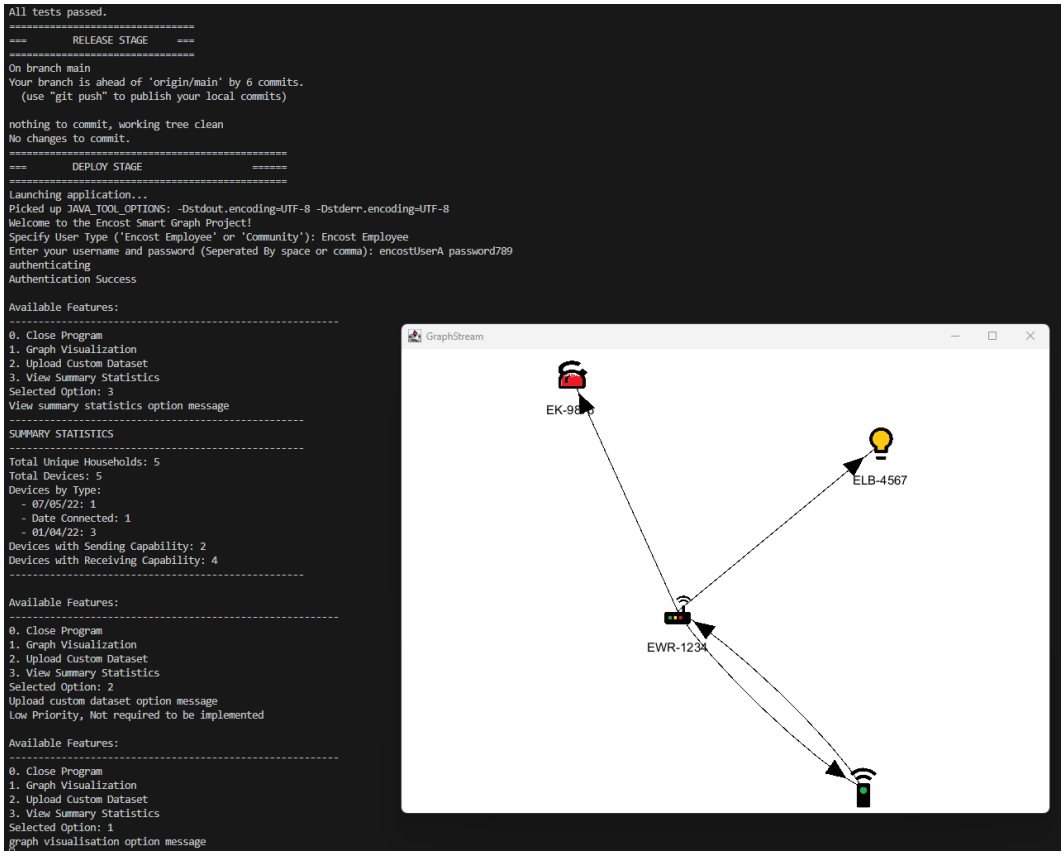

Figure 4.3: CI/CD Pipeline: Deployment stage showing summary statistics output and visualised IoT graph.

Benefits and Alignment

This CI/CD pipeline enforces a critical principle: all test cases must pass before the system is deployed. This ensures:

- Early detection of code errors
- Safer integration of new features
- Improved release quality and confidence
- Reproducibility of builds and tests

5 Conclusion

The Encost Smart Graph Project successfully underwent a series of maintenance tasks aimed at modernizing and improving the system's functionality, reliability, and maintainability.

In Task 1, the legacy GraphStream library was upgraded from version 1.3 to 2.0 to ensure long-term compatibility and support for modern rendering features. This required refactoring deprecated methods and updating property settings to align with the new API. The system was tested and verified to display the IoT device graph correctly without any rendering exceptions.

In Task 2, the incomplete summary statistics feature was fully implemented. The updated system now processes the dataset to extract key insights such as the number of unique households, device types, and communication capabilities. The implementation was validated with JUnit tests, and the output was verified during runtime via the application interface.

In Task 3, a proposed enhancement to allow Community users to view only their own household's graph was assessed and formally rejected. This decision was based on limited project scope, existing user role constraints, and the potential complexity of implementing such fine-grained visualization control. Justification was provided through both technical and design reasoning.

Finally, a **CI/CD pipeline** was implemented using a Windows batch script. This automated pipeline successfully integrated the build, test, and deploy stages and was verified to run end-to-end. All tests passed, and the application launched as expected, demonstrating the effectiveness of the pipeline in supporting continuous delivery.

Overall, the maintenance work improved the robustness of the system, ensured compatibility with future technologies, and upheld code quality through testing and automation.